Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Cloud Foundations & Virtualization — TP5 Partie 2 — Serverless Functions : déployer, sécuriser, superviser et appeler une fonction Azure
4e année Cybersécurité — École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
JIDAL Ilyas
KABORE Mohammad Sharif Jonathan

Filière : Cybersécurité

Année : 2026/2027

Enseignant : M. KARRAKCHOU & M. SOUIDI

Date : 4 novembre 2026

Table des matières

Cadre du TP **2**

1 Prérequis **3**

2 Déployer la fonction dans Azure **3**
2.1 Partie II.1 — Autoriser de nouveau la méthode GET 3
2.2 Partie II.2 — Création de la Function App 4
2.3 Partie II.3 — Ressources créées 5
2.4 Partie II.4 — Déploiement du code 6

3 Tester la fonction déployée **7**
3.1 Partie III.1 — URL de la Function App 7
3.2 Partie III.2 — Accès à la fonction sans clé : erreur 401 7
3.3 Partie III.3 — Niveaux d'autorisation et clés d'accès 8
3.4 Partie III.4 — Récupération de la clé et appel avec la clé 8

4 Superviser la fonction **9**

5 Appeler la fonction depuis un script Python **10**

6 Appeler la fonction depuis une page HTML **11**
6.1 Partie VI.1 — La page HTML 11
6.2 Partie VI.2 — Le problème de CORS 13
6.3 Partie VI.3 — Résultat 14
6.4 Remarque sur la sécurité 14

7 Devoir : déployer, sécuriser, superviser et appeler une fonction HTTP dans le cloud **15**
7.1 Les étapes 15
7.2 Réponses aux questions du sujet 15
7.3 Suppression des ressources 16
7.4 Commandes, fichiers et options utilisés 16

8 Bilan des éléments créés **17**

9 Difficultés rencontrées **17**

Cadre du TP

Niveau	**Cours**	**Plateformes**
4e CY	Cloud Foundations	Visual Studio Code (extension *Azure Functions*) / Portail Azure / Python 3.14 / Microsoft Azure (abonnement *Azure for Students*)
Mode	**Approche**	**Livrable**
Groupe de 2 à 3	Déployer dans Azure la fonction serverless de la Partie 1, la sécuriser par une clé, la superviser, puis l'appeler depuis un script Python et une page HTML	Compte rendu + captures d'écran + code du script et de la page web + suppression des ressources

TABLE 1 – Cadre général du TP

Objectif. La Partie 1 a permis de développer et d'exécuter **en local** la fonction `AddFunction`. Cette seconde partie la fait passer dans le **cloud** : on la déploie dans une *Function App* Azure, on constate qu'elle est protégée par une **clé de fonction** (erreur `401` sans clé), on observe son activité dans *Application Insights*, puis on l'appelle depuis un script Python et depuis une page HTML (API *Fetch*), ce qui oblige à configurer la politique **CORS** de la fonction.

Fil conducteur du TP :

Fonction locale (Partie 1) → Création de la Function App → Déploiement depuis VS Code → Test (erreur `401`, clé de fonction) → Supervision (Application Insights) → Appel Python → Appel HTML (CORS) → Suppression des ressources

Environnement. Poste sous Linux, éditeur Visual Studio Code avec l'extension *Azure Functions*, projet `TP5_crafting` de la Partie 1 (fonction `AddFunction`, modèle de programmation V1, niveau d'autorisation *Function*). Les ressources sont créées dans l'abonnement *Azure for Students* (tenant *Université Internationale de Rabat*), dans la région *Spain Central*. Les tests dans le navigateur sont faits avec Google Chrome. Contrairement à la Partie 1, les ressources de ce TP existent réellement dans Azure : elles sont **supprimées** à la fin (section 7.3).

1 Prérequis

Partie I du TP : disposer d'une fonction qui marche en local.

Le TP déploie la fonction développée en Partie 1. Il faut donc un projet fonctionnel en local : le projet `TP5_crafting` (fonction `AddFunction`, déclencheur HTTP) démarre avec `func start` et répond sur `http://localhost:7071/api/AddFunction`.

2 Déployer la fonction dans Azure

Partie II du TP : autoriser de nouveau la méthode `GET`, créer la Function App*, identifier les ressources créées et déployer le code.*

2.1 Partie II.1 — Autoriser de nouveau la méthode GET

En Partie 1, la liste `methods` de `AddFunction/function.json` avait été réduite à `"post"`. Le sujet demande de remettre `GET`, ce qui permet de tester la fonction déployée depuis la barre d'adresse du navigateur (sections 3 et 6).

Listing 1 – `AddFunction/function.json` avec `GET` et `POST`

```
{
  "scriptFile": "__init__.py",
  "bindings": [
    {
      "authLevel": "function",
      "type": "httpTrigger",
      "direction": "in",
      "name": "req",
      "methods": [
        "get",
        "post"
      ]
    },
    {
      "type": "http",
      "direction": "out",
      "name": "$return"
    }
  ]
}
```

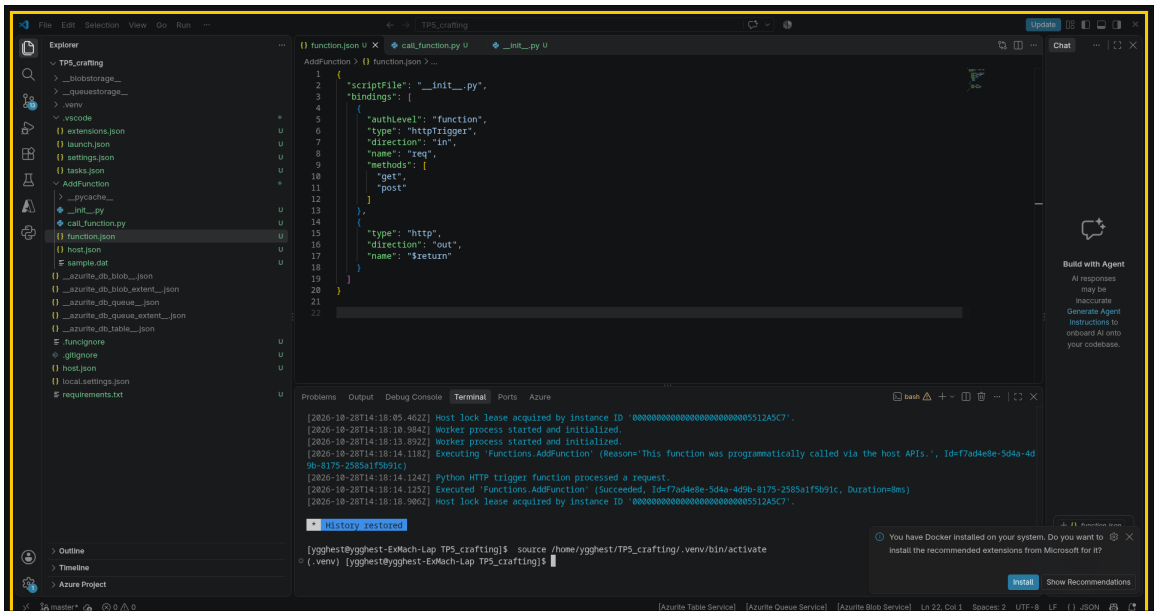


FIGURE 1 – `function.json` dans VS Code : la liste `methods` contient de nouveau `"get"` et `"post"`. Le niveau d'autorisation reste `"function"`.

2.2 Partie II.2 — Création de la Function App

Dans l'icône *Azure* de VS Code, un clic droit sur *Function App* (volet *Resources*) puis *Create new Function App in Azure...* lance l'assistant. Voici les paramètres choisis :

TABLE 2: Paramètres de création de la Function App

Paramètre	**Valeur**	**Explication**
Name	`serverless-func-TP5-pt2`	Nom unique dans tout Azure, car il fait partie de l'URL publique de la fonction
Runtime stack	Python 3.14	Même version que l'environnement local
Location	*Spain Central*	Région Azure utilisée dans les TP précédents
Authentication	*Secrets*	Les accès au stockage et aux services internes se font avec des secrets (chaînes de connexion)

La création prend environ une minute (Figure 2) : l'assistant crée un groupe de ressources, un plan d'hébergement, un compte de stockage, une instance *Application Insights*, puis la *Function App* elle-même.

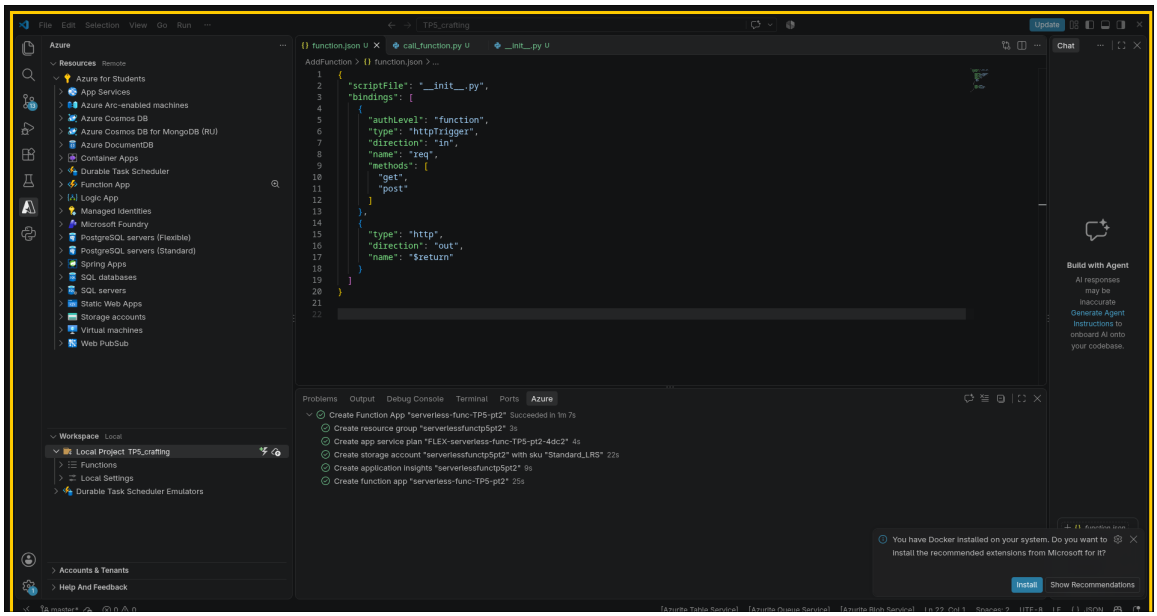


FIGURE 2 – Onglet *Azure* de la sortie de VS Code : *Create Function App "serverless-func-TP5-pt2" Succeeded in 1m 7s*, avec les cinq étapes de création détaillées.

2.3 Partie II.3 — Ressources créées

Le sujet demande de se connecter au portail Azure et d'identifier les ressources créées. La liste *All resources* (Figure 3) montre les ressources suivantes, toutes dans le nouveau groupe de ressources `serverlessfunctp5pt2` :

TABLE 3: Ressources créées par l'assistant

Nom	Type	Rôle
`serverlessfunctp5pt2`	Groupe de ressources	Conteneur logique qui regroupe toutes les ressources du TP : il suffit de le supprimer pour tout effacer
`serverless-func-TP5-pt2`	Function App	Application qui héberge et exécute la fonction `AddFunction`
`FLEX-serverless-func-TP5-pt2-4dc2`	App Service plan	Plan d'hébergement de type *Flex Consumption* : les ressources sont allouées à la demande, et la facturation suit l'usage
`serverlessfunctp5pt2`	Storage account	Compte de stockage (`Standard_LRS`) utilisé en interne par Azure Functions, par exemple pour le code déployé
`serverlessfunctp5pt2`	Application Insights	Service de supervision de la fonction (section 4)

L'instance *Application Insights* est reliée à l'espace de travail *Log Analytics* `workspacetpdocker93e5`, créé lors du TP4 ; les autres ressources de la liste (`banana-classifier-app`, `banana-classifier-instance`,

`managedEnvironment-tpdocker-aa50`) viennent aussi du TP4 et sont dans le groupe `tp_docker`.

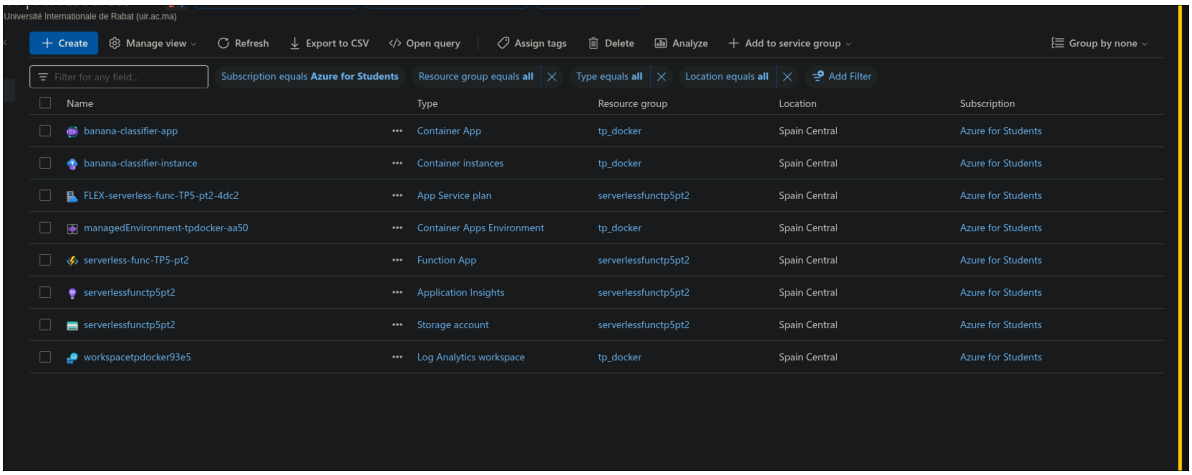


FIGURE 3 – Portail Azure, liste des ressources : les quatre nouvelles ressources (plan *Flex Consumption*, *Function App*, *Application Insights*, *Storage account*) sont dans le groupe `serverlessfunctp5pt2`, région *Spain Central*.

2.4 Partie II.4 — Déploiement du code

Dans le volet *Workspace*, un clic sur l'icône en forme de nuage à côté du projet local `TP5_crafting`, puis le choix de la *Function App* `serverless-func-TP5-pt2`, lance le déploiement. VS Code compresse le dossier du projet, l'envoie à Azure, et la plateforme installe les dépendances de `requirements.txt`.

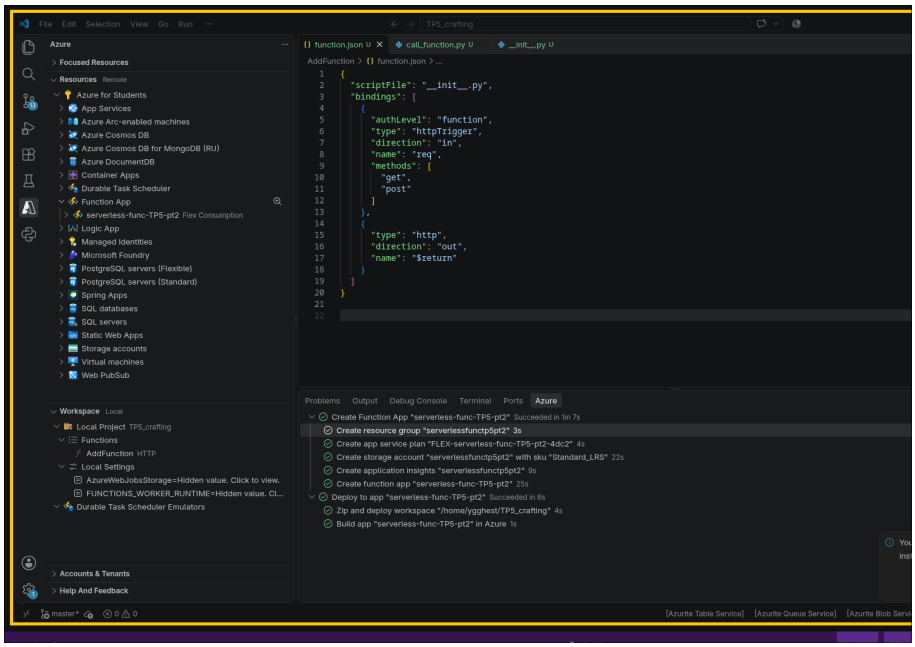


FIGURE 4 – Déploiement réussi : *Deploy to app "serverless-func-TP5-pt2" Succeeded in 6s* (étapes *Zip and deploy workspace* puis *Build app in Azure*). Dans le volet *Resources*, la *Function App* apparaît avec le plan *Flex Consumption* ; dans *Workspace*, la fonction `AddFunction` est de type `HTTP`.

3 Tester la fonction déployée

Partie III du TP : appel depuis le navigateur sans clé, compréhension de l'erreur `401`, puis appel avec la clé de fonction.

3.1 Partie III.1 — URL de la Function App

Dans le portail Azure, la page de la *Function App* donne son URL par défaut (*Default domain*). Elle a la forme :

Listing 2 – URL de la Function App

```
https://serverless-func-tp5-pt2-czc4b3dzg7e7eve5.spaincentral-01.
    azurewebsites.net
```

Ouverte dans le navigateur, cette adresse affiche la page d'accueil d'Azure Functions : *Your Functions 4.0 app is up and running*. Cela montre que l'application tourne, mais pas que la fonction `AddFunction` est accessible : cette page est servie à la racine, pas à l'adresse de la fonction.

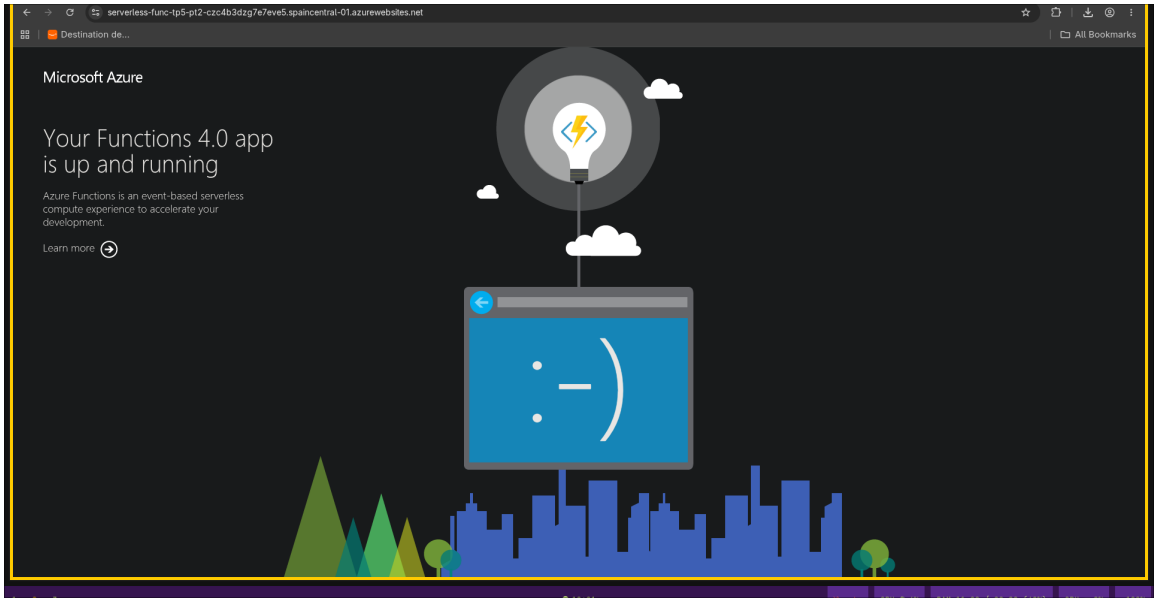


FIGURE 5 – URL racine de la Function App : page d'accueil *Your Functions 4.0 app is up and running*.

3.2 Partie III.2 — Accès à la fonction sans clé : erreur 401

La fonction se trouve à l'adresse de l'application complétée par `/api/addfunction`. Ouverte telle quelle dans le navigateur, elle renvoie une erreur.

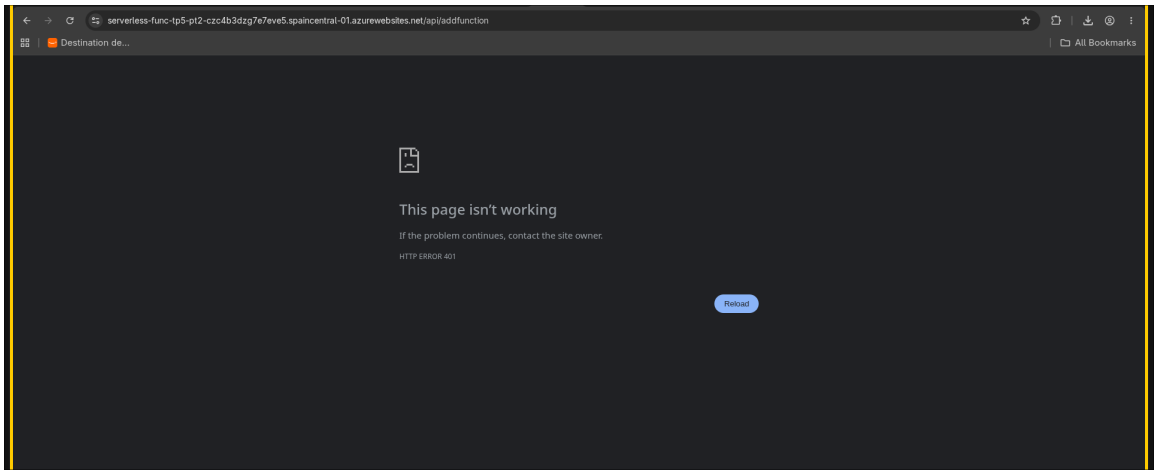


FIGURE 6 – Appel de `/api/addfunction` sans clé : *This page isn't working, HTTP ERROR 401.*

Que signifie l'erreur 401 ? Le code HTTP `401` (*Unauthorized*) signifie que la requête n'est pas authentifiée : le serveur a bien reçu la demande, mais refuse de l'exécuter sans justificatif valide. Ici, la fonction a été créée avec le niveau d'autorisation *Function* (`"authLevel": "function"` dans `function.json`). Ce niveau exige une **clé de fonction** dans chaque requête ; or l'appel du navigateur n'en contient aucune. En local, cette clé n'était pas demandée (Partie 1) ; une fois la fonction dans Azure, elle l'est.

3.3 Partie III.3 — Niveaux d'autorisation et clés d'accès

La documentation Microsoft sur le déclencheur HTTP (sections *Authorization level* et *Function access keys*) indique que le niveau d'autorisation détermine quelle clé doit accompagner la requête :

TABLE 4: Niveaux d'autorisation d'un déclencheur HTTP

Niveau	Exigence
`anonymous`	Aucune clé : n'importe qui peut appeler la fonction
`function`	Une clé propre à la fonction, ou une clé d'hôte, est exigée
`admin`	La clé principale (*master key*) de l'application est exigée

Le niveau choisi étant *Function*, il faut donc **récupérer la clé de la fonction et la joindre à la requête**, soit dans la chaîne de requête (`?code=<clé>`), soit dans l'en-tête `x-functions-key`.

3.4 Partie III.4 — Récupération de la clé et appel avec la clé

Dans le portail Azure, on ouvre la *Function App*, puis la fonction `AddFunction` (en bas de la page *Overview*), puis *Function keys* : la clé `default` est affichée. On l'ajoute à l'URL avec le paramètre `code` :

Listing 3 – URL complète de la fonction (la clé est masquée)

```
1 https://serverless-func-tp5-pt2-czc4b3dzg7e7eve5.spaincentral-01.
    azurewebsites.net/api/addfunction?code=<CLE_DE_FONCTION>
```

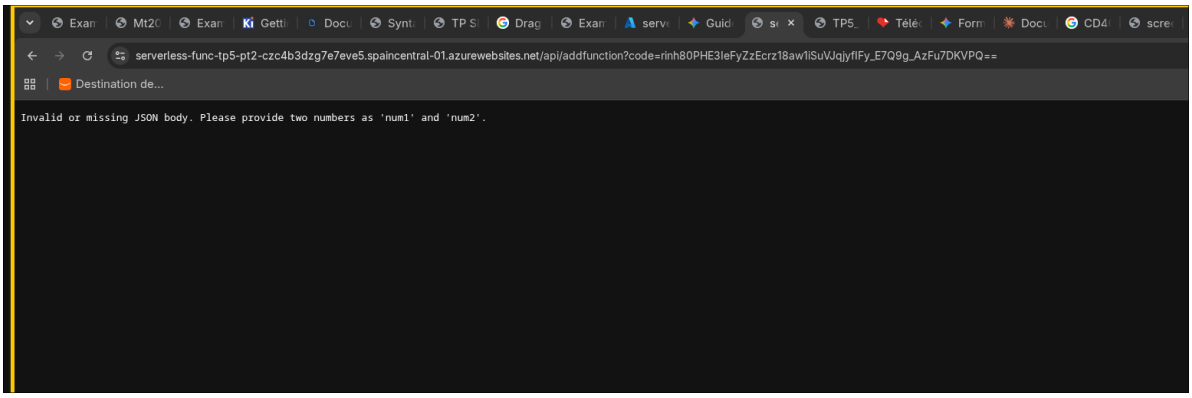


FIGURE 7 – Appel de `/api/addfunction?code=<clé>` dans le navigateur : plus d'erreur `401`, la fonction s'exécute et répond *Invalid or missing JSON body. Please provide two numbers as 'num1' and 'num2'.*

Est-ce que cela marche ? Oui : la fonction est bien exécutée, l'erreur `401` a disparu. Le message affiché vient de la fonction elle-même : le navigateur envoie une requête `GET` sans corps, alors que la fonction attend un corps JSON contenant les deux nombres `num1` et `num2`. La clé a donc rempli son rôle (autorisation), et l'addition elle-même sera testée avec une requête `POST` (sections 5 et 6).

4 Superviser la fonction

Partie IV du TP : observer l'activité de la fonction dans Application Insights.

Dans le portail Azure, on ouvre la ressource *Application Insights* `serverlessfunctp5pt2`, créée avec la *Function App*. La page *Overview* affiche, pour la période choisie (ici la dernière heure), quatre graphiques :

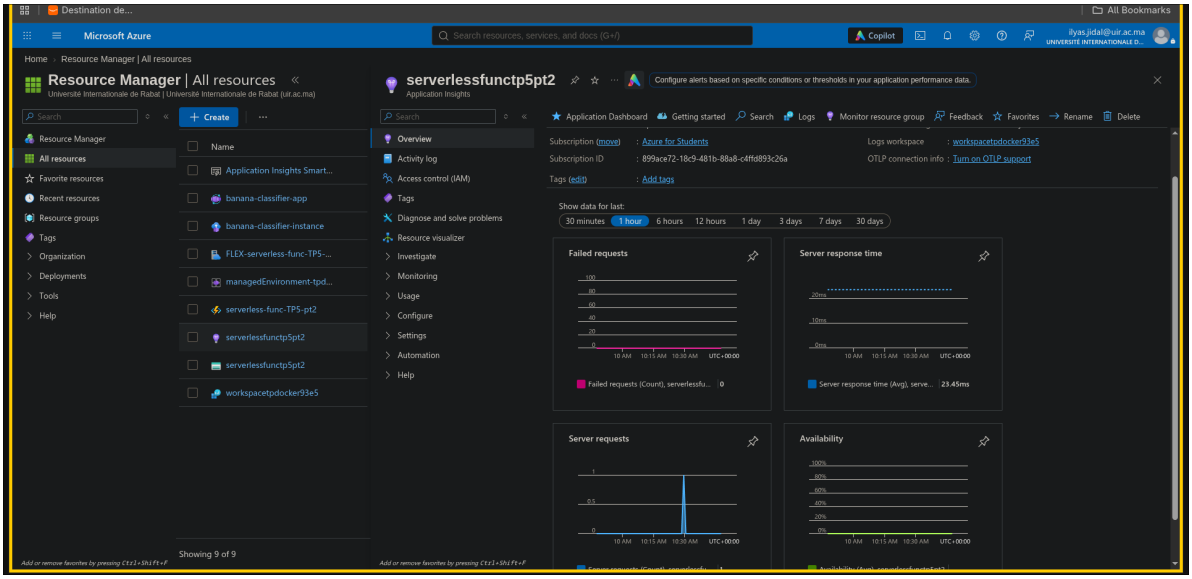


FIGURE 8 – Page *Overview* d'*Application Insights* : *Failed requests*, *Server response time*, *Server requests* et *Availability*. L'espace de travail *Log Analytics* associé est `workspacetpdocker93e5`.

TABLE 5: Lecture des graphiques d'*Application Insights*

Graphique	**Ce qu'il montre**
Failed requests	Nombre de requêtes en échec : `0` sur la période
Server response time	Temps de réponse moyen du serveur : environ 23,45 ms
Server requests	Nombre de requêtes reçues : un pic à `1`, correspondant à l'appel fait avec la clé
Availability	Disponibilité mesurée par des tests de disponibilité ; aucun test n'étant configuré dans ce TP, le graphique ne porte pas de mesure utile

À quoi sert cette ressource ? *Application Insights* est le service de supervision applicative d'Azure Monitor. Il collecte automatiquement la **télémétrie** de la fonction (requêtes, temps de réponse, échecs, exceptions, journaux, dépendances) et la stocke dans l'espace de travail *Log Analytics*. Il permet de vérifier que la fonction répond, de mesurer ses performances, de retrouver la cause d'une erreur dans les journaux (page *Logs*, requêtes en langage KQL) et de déclencher des **alertes** quand un seuil est dépassé. Pour une fonction serverless, où le développeur n'a pas accès au serveur, c'est le principal moyen de voir ce qui se passe.

5 Appeler la fonction depuis un script Python

Partie V du TP : reprendre le script d'appel pour envoyer une requête `POST` à la fonction déployée.

Le script utilise le module `requests` comme celui du TP2 (classifieur d'images). L'URL est celle de la fonction avec sa clé ; le corps JSON contient les deux paramètres attendus, `num1` et `num2`.

Listing 4 – Script d'appel `call_function.py` (fonction déployée dans Azure)

```
import json
import requests

# URL complete de la fonction incluant le code de securite
url = "https://serverless-func-tp5-pt2-czc4b3dzg7e7eve5.
   spaincentral-01.azurewebsites.net/api/addfunction?code=<
   CLE_DE_FONCTION>"

# Donnees JSON a envoyer (les deux nombres pour l'addition)
payload = {"num1": 5, "num2": 10}

# Envoi de la requete POST
response = requests.post(url, json=payload)

# Affichage du resultat
print(f"Code statut : {response.status_code}")
print(f"Reponse : {response.text}")
```

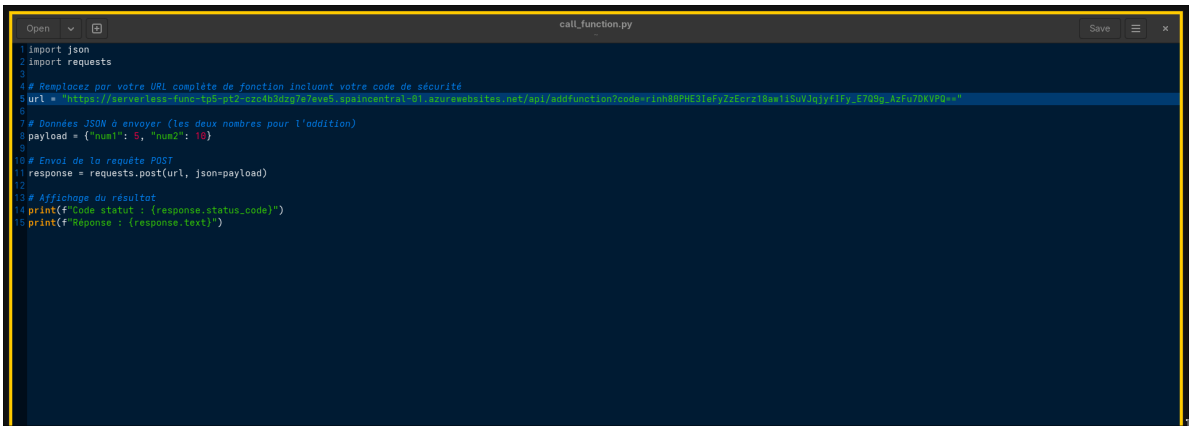


FIGURE 9 – Script `call_function.py` ouvert dans l'éditeur : URL de la fonction avec le paramètre `code`, corps JSON `{"num1": 5, "num2": 10}` et requête `requests.post`.

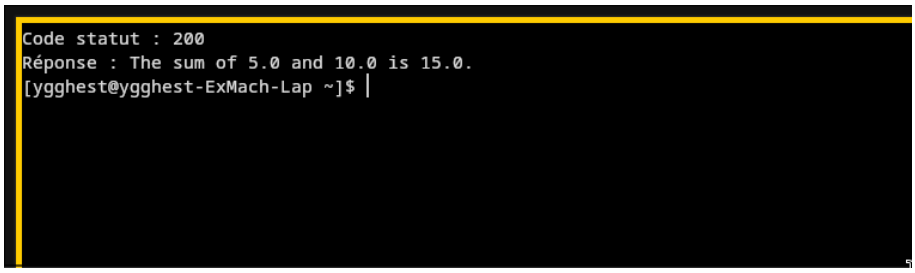


FIGURE 10 – Exécution du script : `Code statut : 200` et `Réponse : The sum of 5.0 and 10.0 is 15.0.`

Le code `200` et la somme `15.0` montrent que la fonction déployée a accepté la clé, lu le corps JSON et calculé l'addition. L'option `json=payload` convertit le dictionnaire en JSON et ajoute l'en-tête `Content-Type: application/json`.

6 Appeler la fonction depuis une page HTML

Partie VI du TP : page web envoyant une requête `POST` avec l'API Fetch*, et configuration de la politique CORS.*

6.1 Partie VI.1 — La page HTML

La page contient deux champs numériques, un bouton et une zone de résultat. Au clic, la fonction JavaScript `calculer()` envoie les deux nombres en JSON par `fetch` et affiche la réponse.

Listing 5 – Page `index.html`

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Test Azure Function - TP5</title>
    <style>
```

```
        body { font-family: Arial, sans-serif; margin: 50px; }
        .container { max-width: 400px; padding: 20px; border: 1px solid #ccc; border-radius: 5px; }
        input, button { margin: 10px 0; padding: 8px; width: 100%; box-sizing: border-box; }
        button { background-color: #0078D4; color: white; border: none; cursor: pointer; }
        button:hover { background-color: #005a9e; }
        #result { margin-top: 15px; font-weight: bold; }
    </style>
</head>
<body>

<div class="container">
    <h2>Addition Serverless</h2>
    <label for="num1">Nombre 1 :</label>
    <input type="number" id="num1" placeholder="Ex: 5">

    <label for="num2">Nombre 2 :</label>
    <input type="number" id="num2" placeholder="Ex: 10">

    <button onclick="calculer()">Calculer</button>

    <div id="result"></div>
</div>

<script>
async function calculer() {
    const num1 = document.getElementById('num1').value;
    const num2 = document.getElementById('num2').value;
    const resultDiv = document.getElementById('result');

    // URL complete Azure incluant le parametre ?code=...
    const url = "https://serverless-func-tp5-pt2-czc4b3dzg7e7eve5.spaincentral-01.azurewebsites.net/api/addfunction?code=<CLE_DE_FONCTION>";

    try {
        const response = await fetch(url, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({ num1: Number(num1), num2: Number(num2) })
        });

        const data = await response.text();
        resultDiv.style.color = "green";
        resultDiv.innerText = "Resultat : " + data;
    } catch (error) {
```

```
52          resultDiv.style.color = "red";
53          resultDiv.innerText = "Erreur␣:␣" + error;
54      }
55  }
56  </script>
57
58  </body>
59  </html>
```

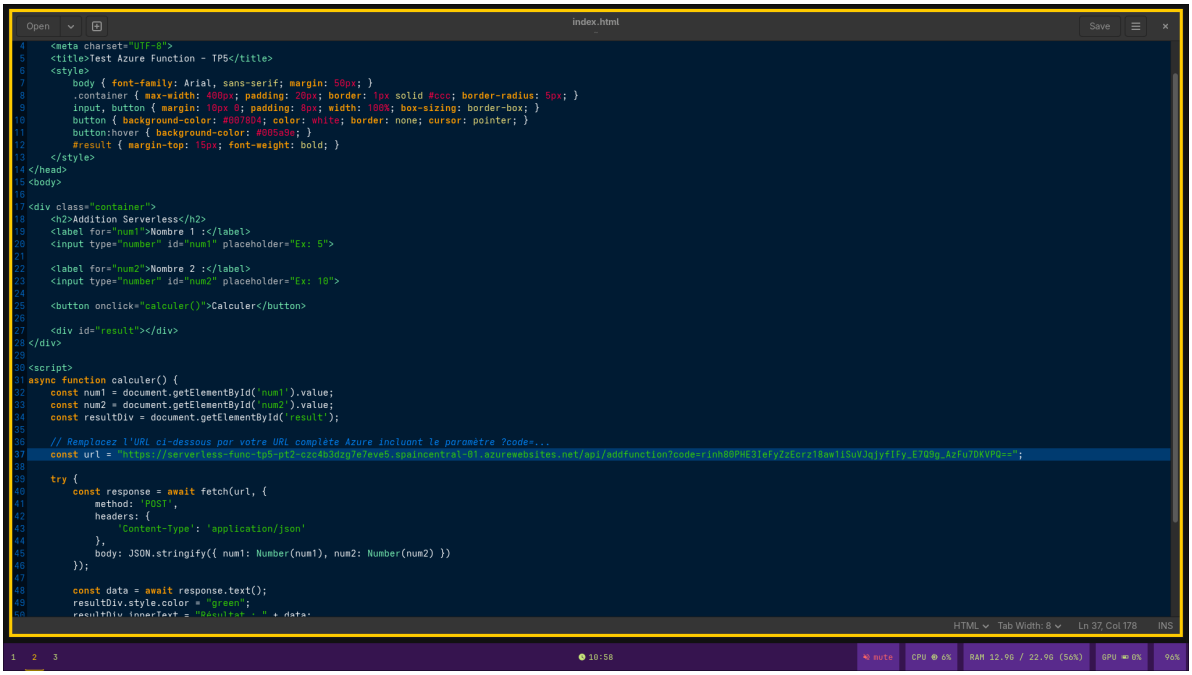


FIGURE 11 – Code de `index.html` dans l'éditeur : fonction `calculer()`, URL de la fonction avec le paramètre `code` et appel `fetch` en `POST` avec un corps JSON.

6.2 Partie VI.2 — Le problème de CORS

La page est ouverte directement depuis le disque (`file:///home/.../index.html`) : son origine n'est donc pas celle de la *Function App*. Le navigateur applique alors la politique de même origine : il envoie la requête, mais refuse de transmettre la réponse au JavaScript de la page si le serveur n'a pas déclaré, par des en-têtes `Access-Control-Allow-Origin`, qu'il autorise cette origine. L'erreur apparaît dans la console des outils de développement de Chrome (onglet *Console*, message *blocked by CORS policy*), et le champ de résultat de la page affiche l'erreur.

Résolution. *CORS* (*Cross-Origin Resource Sharing*) est un réglage de la *Function App*. Dans le portail Azure, on ouvre la *Function App*, puis *API* → *CORS*, et on ajoute l'origine `*` dans *Allowed Origins* (case *Enable Access-Control-Allow-Credentials* décochée), puis on enregistre avec *Save*. Le caractère `*` autorise toutes les origines, y compris une page ouverte depuis un fichier local, comme le demande le sujet.

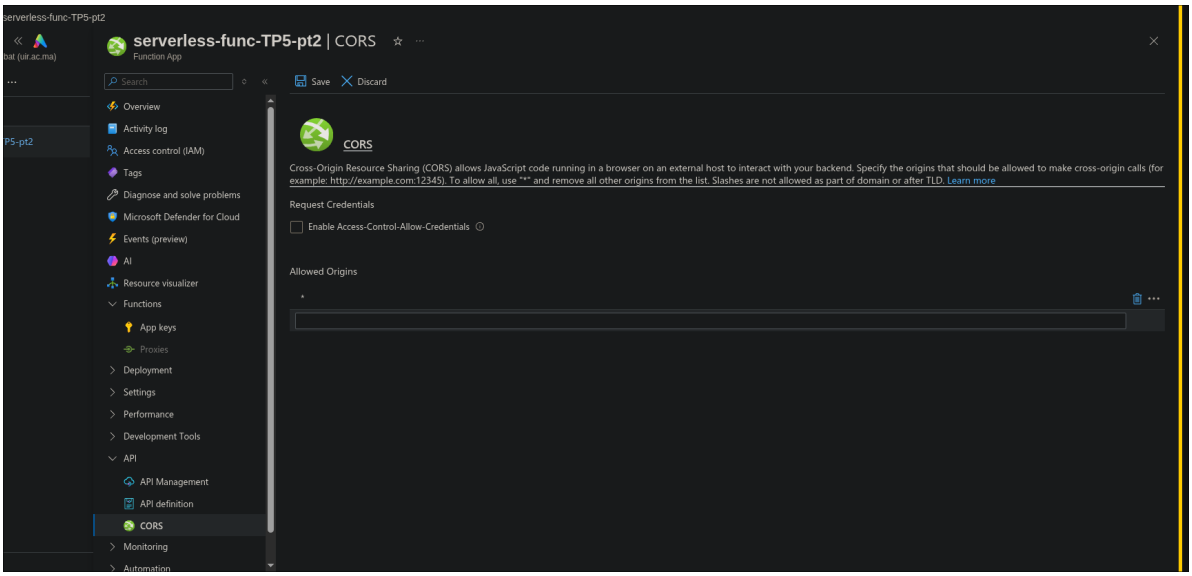


FIGURE 12 – Page *CORS* de la *Function App* `serverless-func-TP5-pt2` : l'origine `*` est présente dans *Allowed Origins*.

6.3 Partie VI.3 — Résultat

Après l'enregistrement du réglage CORS, la page est rechargée. En saisissant 69 et 54 puis en cliquant sur *Calculer*, la zone de résultat affiche la somme renvoyée par la fonction.

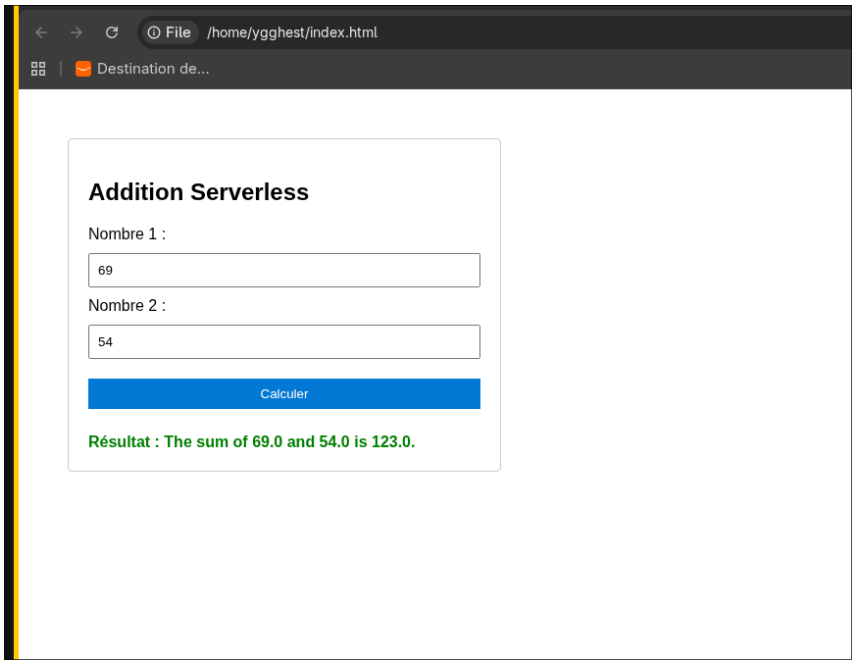


FIGURE 13 – Page `index.html` dans Chrome : *Résultat : The sum of 69.0 and 54.0 is 123.0.*

6.4 Remarque sur la sécurité

La clé de fonction se trouve dans le code de la page : toute personne qui ouvre le code source ou les outils de développement du navigateur la voit et peut s'en servir pour

appeler la fonction. La clé est aussi présente dans l'URL, donc dans l'historique et dans les journaux. Une clé de fonction protège donc un appel de *serveur à serveur* (comme le script Python), mais pas une page publique. De même, `*` dans CORS est pratique pour un TP, mais en production on liste les origines autorisées, et on remplace la clé par une authentification (par exemple Microsoft Entra ID) ou par un intermédiaire côté serveur.

7 Devoir : déployer, sécuriser, superviser et appeler une fonction HTTP dans le cloud

7.1 Les étapes

1. **Partir d'une fonction qui marche en local** (Partie 1) et **réautoriser la méthode** `GET` dans `function.json` (Figure 1).
2. **Créer la Function App** dans Azure depuis VS Code (nom unique, Python 3.14, région, *Secrets*), ce qui crée le groupe de ressources, le plan, le compte de stockage, *Application Insights* et la *Function App* (Figures 2 et 3).
3. **Déployer le code** avec l'icône nuage du volet *Workspace* (Figure 4).
4. **Tester** : la racine de l'application répond, mais `/api/addfunction` sans clé donne `401` (Figures 5 et 6).
5. **Sécuriser par clé** : récupérer la clé de fonction dans le portail (*Function keys*) et l'ajouter à l'URL avec `?code=` (Figure 7).
6. **Superviser** avec *Application Insights* : requêtes, échecs, temps de réponse (Figure 8).
7. **Appeler depuis un script Python** : `requests.post` avec l'URL complète et un corps JSON (Figures 9 et 10).
8. **Appeler depuis une page HTML** avec `fetch` en `POST`, après avoir autorisé les origines dans *CORS* (Figures 11, 12 et 13).
9. **Supprimer les ressources** Azure à la fin du TP (section 7.3).

7.2 Réponses aux questions du sujet

TABLE 6: Réponses aux questions des sections II, III et VI

Question	**Réponse**
Quelles ressources ont été créées dans Azure ? (II)	Un groupe de ressources, un plan *Flex Consumption*, un compte de stockage, une instance *Application Insights* et la *Function App* (Figure 3)
Que voit-on à l'URL de la Function App ? (III)	La page d'accueil *Your Functions 4.0 app is up and running* (Figure 5)
Quelle erreur HTTP obtient-on sur l'URL complète de la fonction, et que signifie-t-elle ? (III)	`401 Unauthorized` : la fonction exige une clé (niveau *Function*) et la requête n'en contient pas (Figure 6)

Question	Réponse
Que faut-il faire d'après la documentation ? (III)	Joindre la clé de fonction à la requête, par `?code=<clé>` ou par l'en-tête `x-functions-key`
L'accès avec la clé fonctionne-t-il ? (III)	Oui : plus d'erreur `401`, la fonction répond (Figure 7)
Que voit-on dans *Application Insights*, et à quoi sert-il ? (IV)	Requêtes, échecs, temps de réponse et disponibilité ; il collecte la télémétrie de la fonction pour la superviser et déclencher des alertes (Figure 8)
Pourquoi la page HTML échoue-t-elle au départ, et comment corriger ? (VI)	Le navigateur bloque la réponse à cause de la politique CORS ; on ajoute l'origine `*` dans *API* → *CORS* de la *Function App* (Figure 12)

7.3 Suppression des ressources

Les ressources Azure sont facturées à l'usage et ne servent plus une fois le TP terminé : le sujet demande de les supprimer. Tous les éléments de ce TP étant dans le groupe de ressources `serverlessfunctp5pt2`, il suffit de supprimer ce groupe, ce qui efface en une fois la *Function App*, le plan, le compte de stockage et *Application Insights*. Cela peut se faire dans le portail (*Resource groups* → `serverlessfunctp5pt2` → *Delete resource group*) ou en ligne de commande :

Listing 6 – Suppression du groupe de ressources du TP

```
az group delete --name serverlessfunctp5pt2 --yes --no-wait
```

Le groupe `tp_docker` et l'espace de travail *Log Analytics* qu'*Application Insights* utilisait viennent du TP4 et ne sont pas dans ce groupe.

7.4 Commandes, fichiers et options utilisés

TABLE 7: Commandes, fichiers et options utilisés dans le TP

Commande, fichier ou option	Rôle
Create new Function App in Azure...	Assistant de VS Code qui crée la *Function App* et les ressources associées
Icône nuage (*Deploy to Function App*)	Envoie le code du projet dans la *Function App*
`function.json` (`authLevel`, `methods`)	Niveau d'autorisation et méthodes HTTP acceptées
`/api/addfunction`	Chemin de la fonction, ajouté à l'URL de l'application
`?code=<clé>` / `x-functions-key`	Deux manières de transmettre la clé de fonction
Function keys	Page du portail où l'on lit la clé `default` de la fonction
Application Insights	Supervision de la fonction (requêtes, échecs, temps de réponse)

Commande, fichier ou option	Rôle
API → CORS	Origines autorisées à appeler la fonction depuis un navigateur
`requests.post(url, json=...)`	Appel de la fonction en `POST` avec un corps JSON (Python)
`fetch(url, {method: 'POST', ...})`	Appel de la fonction en `POST` avec un corps JSON (JavaScript)
`az group delete`	Supprime un groupe de ressources et tout son contenu

8 Bilan des éléments créés

Table 8: Éléments créés pendant le TP5 Partie 2

Élément	Rôle
Groupe de ressources `serverlessfunctp5pt2`	Regroupe toutes les ressources du TP
Function App `serverless-func-TP5-pt2`	Héberge la fonction `AddFunction` dans Azure
Plan `FLEX-serverless-func-TP5-pt2-4dc2`	Plan d'hébergement *Flex Consumption*
Compte de stockage `serverlessfunctp5pt2`	Stockage interne utilisé par Azure Functions
Application Insights `serverlessfunctp5pt2`	Supervision de la fonction
URL `/api/addfunction?code=<clé>`	Point d'entrée de la fonction dans le cloud
Script `call_function.py`	Appelle la fonction déployée en `POST` avec un corps JSON
Page `index.html`	Appelle la fonction déployée avec `fetch` et affiche le résultat
Origine `*` dans CORS	Autorise la page HTML à lire la réponse de la fonction

9 Difficultés rencontrées

Table 9: Problèmes rencontrés et solutions

Problème	Cause	Solution
HTTP ERROR 401 en ouvrant `/api/addfunction` dans le navigateur	Le niveau d'autorisation *Function* exige une clé, absente de la requête	Récupérer la clé `default` dans *Function keys* et l'ajouter avec `?code=`
La fonction ne répond pas à une requête `GET` comme en Partie 1	`GET` avait été retiré de `function.json`	Remettre `"get"` dans la liste `methods` avant le déploiement

Problème	**Cause**	**Solution**
Le navigateur affiche *Invalid or missing JSON body* avec la clé	Le navigateur envoie un `GET` sans corps JSON, alors que la fonction attend `num1` et `num2`	Appeler la fonction en `POST` avec un corps JSON (script Python ou page HTML)
La page HTML n'affiche pas le résultat	La page, ouverte depuis un fichier local, est bloquée par la politique CORS de la *Function App*	Ajouter l'origine `*` dans *API* → *CORS* et enregistrer
La clé de fonction est visible dans le code de la page HTML et dans l'URL	Une clé de fonction est un secret partagé, transmis par le client	À réserver aux appels de serveur à serveur ; en production, utiliser une authentification ou un intermédiaire côté serveur